

EXPLORATION OF REAL-TIME POWER ELECTRONIC SIMULATION ON AMD NPU

A Dissertation
Presented to
the Graduate School of
Clemson University

In Partial Fulfillment
of the Requirements for the Degree
Master of Science
Electrical Engineering

by
Shouyu Du
May 2026

Accepted by:
Dr. Tao Wei, Committee Chair
Dr. Fatemeh Afghah
Dr. Jon Cameron Calhoun

Plain Language Abstract

As the world transitions toward electric vehicles (EVs) and high-performance cloud computing, the power systems inside these technologies are becoming increasingly complex. To ensure these systems are safe and efficient, engineers use a testing method called Hardware-in-the-Loop (HIL), which simulates the behavior of power circuits in real-time. Currently, these simulations rely on specialized computer chips called FPGAs, which are fast but notoriously difficult and expensive to program.

This research explores a new approach: using “AI chips” (specifically AMD’s AI Engines), which are normally designed for artificial intelligence tasks, to run these physics simulations instead. The study developed a method to translate electrical circuit diagrams into mathematical models that these AI chips can process efficiently.

By testing a common power converter circuit used in electronics, this project found that AI chips can indeed run simulations fast enough to keep up with real-world speeds—completing calculations in half a microsecond—provided the data is stored directly on the chip. While there are still limitations regarding how data is moved between memory and the processor, this work proves that AI hardware can be repurposed to help design the next generation of power electronics, offering an easier-to-use alternative to current technologies.

Abstract

The rapid electrification of the automotive and data center sectors has created a critical demand for high-fidelity, real-time simulation of complex power electronic systems. While Hardware-in-the-Loop (HIL) simulation on Field-Programmable Gate Arrays (FPGAs) is the current industry standard, it presents significant challenges regarding development complexity and memory limitations. This dissertation investigates the feasibility of utilizing Neural Processing Units (NPUs)—specifically the AMD AI Engine (AIE) XDNA2 architecture—as a novel platform for real-time, deterministic circuit simulation.

This study establishes a comprehensive automated modeling framework based on graph theory and Massarini’s method to derive linear state-space representations for arbitrary circuit topologies, capable of handling ideal switching events and Dirac impulses. To evaluate the hardware’s capabilities, a Half-Bridge LLC Resonant Converter was simulated using two distinct memory management architectures: a Matrix Cached implementation and a Runtime Matrix Fetching implementation.

Experimental results conducted on an AMD Ryzen AI 9 HX 370 processor demonstrate that the Matrix Cached implementation achieves a deterministic iteration time of approximately $0.5\ \mu\text{s}$, meeting the stringent real-time requirements for high-frequency power electronics. Conversely, the Runtime Matrix Fetching approach revealed significant bottlenecks due to driver latency and cache misses. The study concludes that while the AIE architecture is currently compute-bound due to emulated floating-point arithmetic, it offers a viable, flexible alternative to FPGAs for simulations that fit within localized memory.

Acknowledgments

I would like to express my deepest gratitude to my advisor, Dr. Tao Wei, for his unwavering support and guidance. He has been instrumental in shaping my research and providing valuable insights throughout my academic journey. I am also grateful to my committee members, Dr. Fatemeh Afghah and Dr. Jon Cameron Calhoun, for their constructive feedback and encouragement. My sincere thanks go to my colleagues and friends for their support and collaboration, which have enriched my research experience and made this journey more enjoyable. I thank the Clemson ECE department for their support, guidance, and resources provided throughout my graduate studies. I am also grateful to the funding agencies and organizations that have supported my research.

Table of Contents

Title Page	i
Plain Language Abstract	ii
Abstract	iii
Acknowledgments	iv
List of Tables	vii
List of Figures	viii
1 Introduction	1
1.1 Motivation	1
1.2 Challenges in Real-Time Simulation	2
1.3 Hardware Architectures for HIL	3
1.4 Contribution	4
2 Related Work	5
2.1 Method for real-time circuit simulation	5
2.2 Hardware Application for NPU in Scientific Computing	6
3 Background	7
3.1 HIL Algorithm for Power Electronics Simulation	7
3.2 AI Engine Array	9
4 Research Design and Methods	12
4.1 Pedagogical Framework and State Space Realization	12
4.2 Topological Representation of Circuit Networks	13
4.3 Automated System Formulation	15
4.4 Handling Topology Changes	21
4.5 Handling Degenerate Networks and Impulsive Modes	22
4.6 Discrete-Time Implementation	25
5 Experiment Setup	27
5.1 Principle	27
5.2 Implementation	29
6 Results	36
6.1 Experiment Setup	36
6.2 Accuracy Evaluation	36
6.3 Performance analysis	37

7	Conclusions and Discussion	41
7.1	Summary of Contributions	41
7.2	Methodological Framework	41
7.3	Implementation Strategies and Performance	42
7.4	Discussion on Architecture and Scalability	42
7.5	Final Remarks and Future Work	43
	Bibliography	45

List of Tables

4.1	Incidence Matrix of the Boost Converter Circuit	14
4.2	Cutset Matrix of the Boost Converter Circuit	15
4.3	F Matrix (Full View)	18
4.4	Matrix at Switch On and Diode Off	20
4.5	Matrix With Columns Swapped for Switch S1 OFF and Diode D1 OFF	21
4.6	Matrix at Switch Off and Diode Off	21
6.1	TABLE I: Average Cycle Breakdown Per Iteration for Key Simulation Operations. .	40

List of Figures

3.1	Illustration of two AI Engine (AIE) columns within the XDNA 2 architecture. The layout highlights key components: compute tiles (CTs) for processing, memory tiles (MTs) for data storage, and shim tiles (STs) that manage I/O. [cite_start]Communication is handled by AXI4 interconnects (gray switch boxes). [cite: 77–80]	10
4.1	Boost converter circuit diagram	13
5.1	Flow chart of the Iteration Process	27
5.2	Circuit diagram of half-bridge LLC converter.	29
5.3	Data flow of the two implementations (left) and column-major MVM illustration (right).	31
5.4	Matrix Cached Pipeline	33
5.5	Runtime Matrix Fetching Pipeline	34
6.1	Output Voltage of Half-Bridge-LLC at x20 oversampling iteration frequency.	37
6.2	L1 Current of Half-Bridge-LLC at x20 oversampling iteration frequency.	38
6.3	Box and Whisker plot showing iteration times for (a) Matrix Cached and (b) Runtime Matrix Fetching implementations. 50 runs per configuration.	39

Chapter 1

Introduction

1.1 Motivation

The global imperative to reduce carbon emissions and mitigate climate change has dramatically accelerated the electrification of the automotive sector. As the market share of Electric Vehicles (EV) and Hybrid-Electric Vehicles (HEV) expands, the internal electronic architectures of these platforms are becoming increasingly sophisticated. Modern powertrain systems rely heavily on complex power electronics, such as Onboard Chargers (OBC) and DC/DC converters (e.g., Dual Active Bridge or CLLC resonant converters), to efficiently manage power flow between high-voltage and low-voltage networks [3, 8, 9, 16].

Beyond the automotive industry, the exponential growth of cloud computing and Artificial Intelligence (AI) workloads has triggered a parallel surge in demand for efficient power distribution within data centers. These facilities now require high-density power conversion stages to handle the massive energy requirements of modern server racks.

This structural complexity, characterized by high-frequency switching and a high density of components, has created a critical demand for faster, safer, and more accurate simulation technologies. Consequently, Hardware-in-the-Loop (HIL) simulation has emerged as the industry standard for system testing and validation. In a HIL configuration, the physical plant is replaced by a real-time computational model. This model ingests the controller's output, computes the plant's dynamic response, and feeds the resulting signals back to the controller. This closed-loop setup facilitates automated, reliable, and reproducible testing under laboratory conditions, significantly mitigating

the costs and safety risks associated with testing on physical prototypes.

1.2 Challenges in Real-Time Simulation

Simulating power electronic circuits in real-time is a computationally demanding task. Unlike mechanical systems, power electronics are “stiff” systems characterized by high eigenvalues and extremely fast dynamics driven by high switching frequencies. To achieve real-time performance, the selection of an appropriate mathematical modeling strategy is paramount.

1.2.1 Modeling Paradigms: Nodal vs. State-Space

Broadly, circuit simulation approaches fall into Nodal Analysis or State-Space Analysis. Nodal analysis (common in SPICE tools) relies on iterative solvers (e.g., Newton-Raphson) to resolve non-linearities. However, the variable execution time of iterative methods makes them unsuitable for the strict, deterministic timing requirements of HIL simulation [14].

In contrast, State-Space analysis describes the system using a set of first-order differential equations:

$$\dot{x}(t) = Ax(t) + Bu(t) \tag{1.1}$$

By discretizing these equations with a fixed time step, the system response can be calculated using deterministic matrix-vector multiplications. This predictability makes state-space methods the preferred choice for real-time applications.

1.2.2 The Topology Explosion

A specific challenge in this domain is the representation of switching events. To maximize computational efficiency, semiconductor devices are often modeled as ideal switches. However, this binary behavior implies that for a topology with n independent switches, there exist 2^n unique circuit topologies.

In a linear state-space formulation, each topology corresponds to a unique set of system matrices (A_α, B_α) . While it is not feasible to calculate these matrices on-the-fly during a sub-microsecond simulation step, they can be precomputed. However, storing a distinct state-space description for every unique combination of switch positions leads to a combinatorial explosion,

imposing severe demands on both memory bandwidth and storage capacity.

1.3 Hardware Architectures for HIL

To address these computational and timing challenges, the industry has largely adopted Field-Programmable Gate Array (FPGA) platforms. FPGAs offer the fine-grained clock scheduling necessary to capture fast switching events that standard CPUs cannot handle efficiently. Current state-of-the-art HIL implementations—such as those by PLECS [3], dSPACE [9], and Typhoon HIL [16]—manage complexity by deploying these linear state-space iteration models onto the FPGA fabric.

However, FPGA-based HIL systems face inherent limitations: development requires complex Hardware Description Language (HDL) expertise, compilation times are lengthy, and there is a rigid trade-off between generic solvers (easy to use but resource-heavy) and circuit-specific implementations (efficient but inflexible). Furthermore, as the number of switches increases, the limited on-chip memory of FPGAs becomes a bottleneck for storing the requisite 2^n state-space models [9].

1.3.1 Emerging Platform: AI Engines (AIE)

With the ubiquity of AI workloads, a new class of hardware known as the Neural Processing Unit (NPU) has emerged. While primarily designed for AI inference, the architecture of these processors—optimized for massive parallelism and high-bandwidth matrix-vector multiplication—makes them a promising, yet unexplored, candidate for real-time HIL simulation.

This thesis investigates the AMD AI Engine (AIE) architecture found in the Versal ACAP series. Unlike the fine-grained logic of FPGAs, the AIE is a coarse-grained array of VLIW (Very Long Instruction Word) and SIMD (Single Instruction, Multiple Data) processors. Because the core mathematical operation of State-Space simulation is matrix multiplication, the AIE’s hardware acceleration aligns theoretically with the needs of power electronics simulation. Exploring whether this dataflow-driven architecture can sustain the low-latency feedback loops required for HIL is a novel research direction.

1.4 Contribution

This study explores, for the first time, the feasibility of deploying a real-time HIL simulation purely on AMD’s AIE architecture. The central question is whether a spatial computing platform designed for AI throughput can meet the sub-microsecond latency requirements of high-frequency switching circuits.

Our main contributions are summarized as follows:

- **Automated Modeling Algorithm:** We provide a detailed step-by-step description of the algorithm used to realize and precompute state-matrices for arbitrary circuit topologies.
- **First Experimental Evaluation:** We present the first documented implementation of a power converter real-time simulation running on AMD AIE hardware.
- **Novel Pipelined Architecture:** We propose a pipelined dual-core architecture with two implementation variants—*Matrix Cached* and *Runtime Matrix Fetching*—to efficiently address the 2^n topology memory challenge on AIE tiles.
- **Comparative Analysis:** We conduct a rigorous accuracy evaluation against C++ (double precision) and PLECS commercial baselines to validate the numerical fidelity of the AIE implementation.
- **Performance Profiling:** We offer a detailed breakdown of execution performance, analyzing Direct Memory Access (DMA) overhead, cache behavior, and vector processing utilization to identify system bottlenecks.

Chapter 2

Related Work

2.1 Method for real-time circuit simulation

The SPICE (Simulation Program with Integrated Circuit Emphasis) simulator, developed in the 1970s at UC Berkeley, has long been the cornerstone of electronic circuit simulation. Over the decades, numerous enhancements have been made to its non-linear device modeling capabilities, establishing it as the gold standard for offline accuracy. However, the core issue of SPICE-based simulators lies in their reliance on Modified Nodal Analysis (MNA) coupled with iterative numerical methods, such as the Newton-Raphson algorithm. While highly accurate, these iterative solvers introduce non-deterministic execution times, rendering them unsuitable for hard real-time applications where a fixed time-step is mandatory.

While there are existing works on deploying SPICE-like simulations on Field Programmable Gate Arrays (FPGAs) to accelerate computation [21], these implementations often suffer from resource bottlenecks when scaling to larger circuit topologies or require complex floating-point handling that limits achievable clock frequencies.

To address the latency constraints of real-time emulation, researchers have historically pivoted toward discretized modeling techniques. However, representing semiconductor switches (e.g., MOSFETs, IGBTs) in these frameworks presents a significant challenge. Non-state-space based methods often struggle with the “switched circuit” problem. For instance, the resistive switch model [5] attempts to model the device as a binary resistance (high/low). However, this approach often results in a stiff system matrix with widely spread eigenvalues, requiring extremely small time-steps

to maintain stability, which is counter-productive for real-time performance.

Alternatively, the inductive-capacitive (LC) switch model [17]—often associated with the Fixed Admittance Matrix (FAM) method—mitigates the matrix re-factorization overhead. Yet, this introduces artificial storage elements into the circuit, which can impair simulation accuracy by generating spurious oscillations during switching events. Consequently, traditional methods largely fail to strike an optimal balance between the high fidelity required for modern power electronics and the low latency required for Hardware-in-the-Loop (HIL) testing.

2.2 Hardware Application for NPU in Scientific Computing

The advent of the Neural Processing Unit (NPU), specifically architectures like AMD’s Versal AI Engines (AIE), marks a paradigm shift in heterogeneous computing. While the AIE architecture is primarily designed for machine learning and Large Language Model (LLM) inference tasks [19, 20], its underlying structure—consisting of arrays of VLIW (Very Long Instruction Word) vector processors with massive memory bandwidth—holds significant distinct potential for deterministic scientific computing.

Despite this potential, there are few prior works exploring the applicability of NPUs outside the domain of neural networks. The challenge lies in the rigidity of the dataflow models required by NPUs, which differ significantly from the general-purpose flexibility of CPUs or the massive thread parallelism of GPUs.

However, recent studies have begun to demonstrate the efficacy of these architectures for rigorous mathematical modeling. For instance, NPUs have been successfully adapted for real-time signal processing [7], leveraging their vector capabilities for fast Fourier transforms and filtering. Furthermore, in the domain of electromagnetic physics, the architecture has been applied to scientific Finite-Difference Time-Domain (FDTD) simulations [23]. These works suggest that the spatial compute architecture of the NPU can be repurposed to solve linear algebra-heavy differential equations, offering a promising avenue for accelerating circuit simulation beyond the capabilities of traditional processors.

Chapter 3

Background

3.1 HIL Algorithm for Power Electronics Simulation

Real-time simulation of power electronic circuits is a significant challenge, especially when aiming to accurately model high-frequency switching converters. To effectively capture the rapid switching dynamics, a sufficient oversampling factor $\kappa = \frac{T_s}{T}$ is required, where T_s denotes the switching period and T the simulation time step [9]. For Hardware-in-the-Loop (HIL) simulation, an oversampling factor of 20 or greater is typically recommended [4]. For instance, if the switching frequency f_s is 100 kHz, the simulation step T must be ≤ 500 ns.

3.1.1 Switch Modeling

While various linear switch modeling exists, such as the resistive switch model [5], or the inductive-capacitive switch model [17], for representing the semiconductor devices. While their advantage requires only a single set of system matrix, but the representation cause high eigen-values and stiff system, which requires very small time step to maintain stability, thus is not suitable for real-time simulation.

In the state-space approach, every semiconductor is replaced by a short circuit in on-state or by an open-circuit in off-state[9, 14, 2, 16]. After the semiconductor are replaced according to their switch-state, only linear devices remain. Then, based on the circuit topology, a set of system matrices (A_α, B_α) can be generated for each unique switch state combination α . This approach

allows the system to be represented as a sequence of linear time-invariant (LTI) systems, which can be efficiently solved using linear state-space iteration on FPGA hardware.

3.1.2 Linear State-Space Iteration

Although the specific implementation details of commercial HIL platforms such as PLECS [3], dSPACE [9], and Typhoon HIL [16] are proprietary, they share a common fundamental approach: utilizing ideal, linear switch models to represent power semiconductors. Unlike physical device models (e.g., SPICE), ideal switches allow the circuit to be represented as a sequence of linear time-invariant (LTI) systems, which can be efficiently solved using linear state-space iteration on FPGA hardware [9].

In this framework, the circuit is modeled using a set of first-order differential equations where the state variables x typically represent inductor currents and capacitor voltages. For any fixed combination of switch states, the system behavior is described by the continuous-time state-space equations:

$$\dot{x} = A_\alpha x + B_\alpha u \quad (3.1)$$

$$y = C_\alpha x + D_\alpha u \quad (3.2)$$

where A_α is the state transition matrix, B_α is the input matrix, C_α is the output matrix, and D_α represents direct feedthrough from input to output.

The core computational task involves discretizing these continuous-time equations using numerical integration schemes, such as the forward Euler, backward Euler, or trapezoidal rule. At each time step, the state vector x is updated based on the current input u and the previous state, requiring matrix-vector multiplications (MVM).

If the circuit contains n switches, there are theoretically up to 2^n distinct topologies [9]. Every unique combination α of switch states (on/off) yields a specific set of system matrices $A_\alpha, B_\alpha, C_\alpha, D_\alpha$. The state-space approach is practical for real-time simulation only if these matrices can be generated automatically or pre-calculated for the required configurations [2].

3.1.3 Handling Discontinuities and Dirac Impulses

A critical complexity in using ideal switches is the handling of instantaneous topological changes. In physical reality, continuity of inductor currents and capacitor voltages is maintained by semiconductor physics. In contrast, ideal switching allows for instantaneous topological changes that may force a state variable to a specific value discontinuously [2, 14]. For example, if a switch shorts a capacitor or opens a series inductor, the state variable is mathematically forced to jump, resulting in theoretical Dirac impulses [4]:

$$[x(t_{sw}^+) - x(t_{sw}^-)] \cdot \sigma(t - t_{sw}) = \int_{-\infty}^t [x(t_{sw}^+) - x(t_{sw}^-)] \cdot \delta(\tau - t_{sw}) d\tau \quad (3.3)$$

where σ is the Heaviside step function, δ is the Dirac delta function, and $x(t_{sw}^-)$ and $x(t_{sw}^+)$ represent the state limits before and after the switching instant t_{sw} .

If not handled numerically, these impulses lead to erroneous switch state detection or simulation instability. To address this, algorithms separate the system output into a non-impulsive component (y) and an impulsive component ($y^{(\delta)}$) [2]:

$$y^{(\delta)} = C^{(\delta)}x + D^{(\delta)}u \quad (3.4)$$

When a switching event occurs, if $y^{(\delta)}$ is non-zero (indicating, for example, a voltage spike across a freewheeling diode caused by an opening inductor), the algorithm detects an inconsistent topology and forces the necessary switch transitions immediately [2, 15, 9].

3.2 AI Engine Array

The AIE (AI Engine) array is a collection of AIE tiles that form the core of the AMD AIE architecture. This architecture combines both Single Instruction Multiple Data (SIMD) and Very Long Instruction Word (VLIW) principles. We use AMD’s AIE XDNA2 architecture [1, 18] as a representative example of spatial computing.

In this architecture, processor cores are organized into a two-dimensional grid structure. Specifically, the XDNA2 consists of 8 vertical columns, each containing a fixed layout of tiles: four compute tiles (CTs) for processing, one memory tile (MT) for data storage, and one shim tile (ST)

for interfacing with external components. Figure 3.1 illustrates the structural pattern of the array.

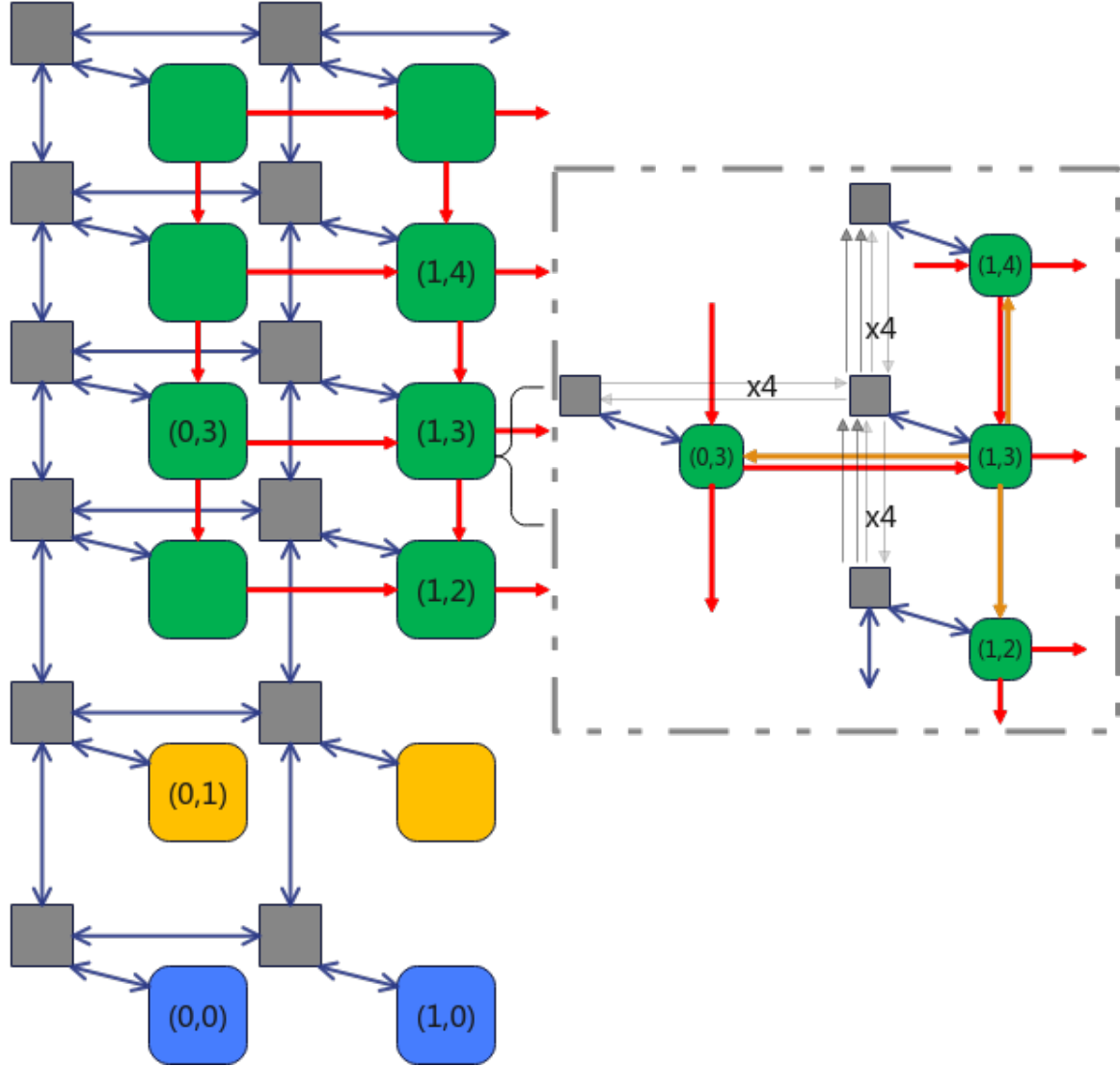


Figure 3.1: Illustration of two AI Engine (AIE) columns within the XDNA 2 architecture. The layout highlights key components: **compute tiles (CTs)** for processing, **memory tiles (MTs)** for data storage, and **shim tiles (STs)** that manage I/O. [cite_start]Communication is handled by AXI4 interconnects (gray switch boxes). [cite: 77–80]

3.2.0.1 Data Movement

The AIE offers a highly flexible dataflow programming model. The switch boxes (gray boxes in Figure 3.1) provide four or more 64-bit AXI4-Stream paths in all four directions. Additionally, the cascade stream interface (red arrows) connects each CT with a 512-bit wide path, and each CT

has 256-bit direct memory access to its neighbors (yellow arrows).

3.2.0.2 Compute Tiles (CT)

Each compute tile serves as the computation core, consisting of a 32-bit scalar RISC processor and a VLIW vector processor. The vector processor in XDNA2 operates at up to 1.8 GHz and supports SIMD operations on int8, int16, and bfloat16 datatypes, with emulation support for single-precision floating-point (FP32). Due to the lack of cache and branch prediction on the processors, all instructions have fixed latencies, requiring the compiler to statically schedule instructions to avoid pipeline hazards.

3.2.1 Programming the Platform

Programming the platform involves two main components: kernel codes mapped to execute on CTs and the Multi-Level Intermediate Representation (MLIR) [12], which specifies data flow and buffer allocations between tiles [22].

Chapter 4

Research Design and Methods

This section describes the theory and methodology used in realizing the state-space iteration for Hardware-in-the-Loop (HIL) simulation.

4.1 Pedagogical Framework and State Space Realization

While the specific state-space HIL algorithms employed by commercial platforms such as PLECS [3], dSPACE [9], and Typhoon [16] remain proprietary, they generally share a common approach: modeling power semiconductor devices as ideal, linear switches. Furthermore, these platforms typically convert circuit topologies into linear state-space difference equations prior to FPGA deployment.

This work presents a comparable linear state-space algorithm tailored for the HIL simulation of power electronic circuits.

Pedagogical Note: To facilitate the derivation of the mathematical framework, we analyze a simple **Boost Converter** topology (Figure 4.1) in this chapter. However, the experimental validation chapter utilizes a more complex **Half-Bridge LLC Resonant Converter**. The methodology derived here is generalizable to the LLC topology.

The Boost circuit comprises an inductor L , capacitor C , diode D , switch S , and load resistor R . The switch S is controlled by a Pulse-Width Modulation (PWM) signal with a switching period T_s and duty cycle D . Selecting the inductor current i_L and capacitor voltage v_C as state variables, the component dynamics are governed by the constitutive equations $i_C = C \frac{dv_C}{dt}$ and $v_L = L \frac{di_L}{dt}$.

The system is driven by the input source voltage V_{in} .

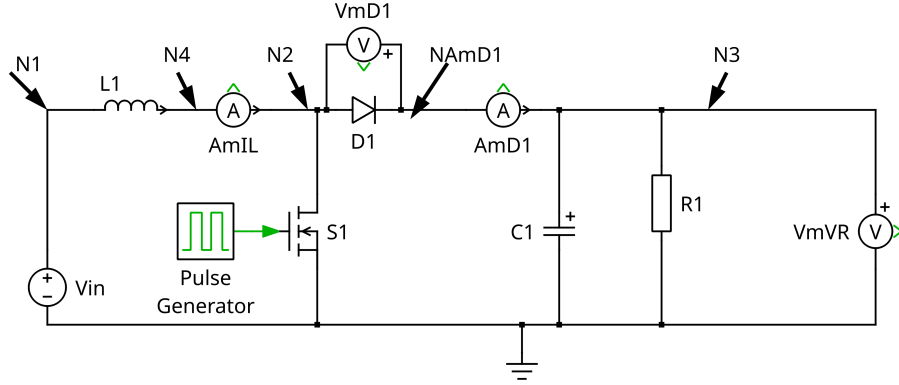


Figure 4.1: Boost converter circuit diagram

4.2 Topological Representation of Circuit Networks

To realize the state-space iteration, the physical circuit must first be translated into a numerical format suitable for linear algebra operations. This is achieved via graph theoretic matrices.

4.2.1 Incidence Matrix

The behavior of any circuit network is governed by three fundamental laws: Kirchhoff's Current Law (KCL), Kirchhoff's Voltage Law (KVL), and the specific element branch characteristics. To apply these laws within a computational framework, the visual topology of the circuit's directed graph G_d must be captured by the **node-branch incidence matrix**.

Definition: Incidence Matrix A_a . For a directed graph G_d consisting of n nodes and b branches, the incidence matrix is defined as an $n \times b$ matrix:

$$A_a = [a_{ij}]$$

where the entries are determined by the orientation of the branches relative to the nodes [6]:

$$a_{ij} = \begin{cases} 1 & \text{if branch } j \text{ is incident at node } i \text{ and directed **away** from the node} \\ -1 & \text{if branch } j \text{ is incident at node } i \text{ and directed **toward** the node} \\ 0 & \text{if branch } j \text{ is not incident at node } i \end{cases}$$

Applying this definition to the boost converter circuit in Figure 4.1, we derive the incidence matrix shown in Table 4.1. Note that additional branches (such as ammeters and voltmeters) are included in the topological description.

Table 4.1: Incidence Matrix of the Boost Converter Circuit

Node	Vin	S1	L1	D1	AMD1	VMD1	C1	R1	VM-VR	AM-IL
N1	1	0	1	0	0	0	0	0	0	0
N2	0	1	0	1	0	1	0	0	0	-1
N3	0	0	0	0	-1	0	1	1	1	0
N4	0	0	-1	0	0	0	0	0	0	1
NAMD1	0	0	0	-1	1	-1	0	0	0	0

The matrix is of shape $(n \times b)$. It should contain n columns of linearly independent vectors, which can be realized by finding the pivots of the matrix. The incidence matrix columns are then rearranged to have the first n columns be linearly independent. The incidence matrix of the boost converter circuit happens to already have the first n columns linearly independent, thus no rearrangement is needed.

4.2.2 Cutset Matrix

In graph network theory, an edge cutset is any set of edges whose removal splits a connected graph into two disconnected components. On page 140 of [6], the cutset matrix is defined as follows:

Definition: Cutset Matrix D_a . For a directed graph G_a with b branches and n_c oriented cutsets, we define the cutset matrix to be an $n_c \times b$ matrix:

$$D_a = [d_{ij}]$$

where

$$d_{ij} = \begin{cases} 1 & \text{if branch } j \text{ is in cutset } i, \text{ and their directions agree} \\ -1 & \text{if branch } j \text{ is in cutset } i, \text{ and their directions oppose} \\ 0 & \text{if branch } j \text{ is not in cutset } i \end{cases}$$

While manual identification is possible, a more systematic approach leveraging the incidence matrix is derived in [6], given by:

$$D = A_T^{-1} A$$

where A_T is the tree component of the incidence matrix A .

Theorem 3-2 of [6] states that any n linearly independent rows of the incidence matrix A form a tree of A . Thus, by selecting the first n rows of the incidence matrix A of the boost converter circuit, we can form a tree of A and compute the cutset matrix D .

The resulting cutset matrix is presented in Table 4.2. The first n columns of the cutset matrix is the tree component, while the remaining columns (colored in yellow or green) correspond to the cotree component.

Table 4.2: Cutset Matrix of the Boost Converter Circuit

Node	Vin	S1	L1	D1	AMD1	R1	VMD1	C1	VM-VR	AM-IL
N1	1	0	0	0	0	0	0	0	0	1
N2	0	1	0	0	0	1	0	1	1	-1
N3	0	0	1	0	0	0	0	0	0	-1
N4	0	0	0	1	0	-1	1	-1	-1	0
NAMD1	0	0	0	0	1	-1	0	-1	-1	0

4.3 Automated System Formulation

4.3.1 Classification of Port and Non-Port Elements

Following the framework established by Chua [6] and Massarini [14], network elements are categorized into *port* and *non-port* elements. Port elements represent the primary dynamic or interface components of the network and are defined to include:

1. All independent voltage and current sources.
2. All ideal switches and diodes.

3. All capacitors and inductors.
4. Output variable sensors (specifically short-circuit elements like ammeters and open-circuit elements like voltmeters), provided they are distinct from the port elements listed above.

All remaining components are classified as non-port elements.

4.3.2 Method of Systematic Elimination

To apply the method of systematic elimination, the fundamental cutset matrix is partitioned to reflect both the topological structure (tree vs. cotree) and the functional classification (port vs. non-port).

We define the following subscripts to denote specific branch types:

- a, b : Port elements located in the tree and cotree, respectively.
- z, y : Non-port elements located in the tree and cotree, respectively.

Under these definitions, the Fundamental Cutset Equations (KCL) are formulated as:

$$\left[\begin{array}{cc|cc} \text{Tree} & & \text{Cotree} & \\ \hline \mathbf{1}_{aa} & \mathbf{0}_{az} & \mathbf{D}_{ay} & \mathbf{D}_{ab} \\ \mathbf{0}_{za} & \mathbf{1}_{zz} & \mathbf{D}_{zy} & \mathbf{D}_{zb} \end{array} \right] \begin{bmatrix} i_a \\ i_z \\ i_y \\ i_b \end{bmatrix} = \begin{bmatrix} \mathbf{0} \\ \mathbf{0} \end{bmatrix} \quad (4.1)$$

This partition explicitly isolates the port variables (a, b) from the non-port variables (z, y) .

For the boost converter circuit under analysis, the resistor R_1 is identified as the sole non-port element. The resulting classification of elements into the topological sub-matrices is detailed below:

Tree Port Elements (a) = $\{V_{in}, S_1, L_1, D_1, AMD_1\}$

Non-Port Elements (z) = $\emptyset$

Cotree Port Elements (b) = $\{VMD_1, C_1, VMVR, AMIL\}$

Non-Port Elements (y) = $\{R_1\}$

For the boost converter circuit, given there is no non-port element in the tree, the sub-matrices $\mathbf{D}_{zy}$ and $\mathbf{D}_{zb}$ are empty. Thus, the D_{ay} and D_{ab} sub-matrices can be directly extracted from the cutset matrix, which correspond to the orange and green sections shown in Table 4.2.

The systematic elimination process begins by forming a tableau form (see page 261 of [6]):

$$\begin{bmatrix} 1 & 0 & D_{zy} & 0 & 0 & 0 & D_{zb} & 0 \\ 0 & 1 & 0 & -D_{zy}^t & 0 & 0 & 0 & -D_{ay}^t \\ 0 & 0 & D_{ay} & 0 & 1 & 0 & D_{ab} & 0 \\ 0 & 0 & 0 & -D_{zb}^t & 0 & 1 & 0 & -D_{ab}^t \\ F_{iz} & F_{vy} & F_{iy} & F_{vz} & F_{ia} & F_{vb} & F_{ib} & F_{va} \end{bmatrix} \begin{bmatrix} i_z \\ v_y \\ i_y \\ v_z \\ i_a \\ v_b \\ i_b \\ v_a \end{bmatrix} = \mathbf{0} \quad (4.2)$$

Note that the first and third rows in equation 4.2 are the KCL equations, the second and fourth rows are KVL equations, and the last row gives the non-port branch characteristics.

For the boost converter circuit, the right vector variable is defined as: $i_z : \emptyset$, $v_y : \{V_{R1}\}$, $i_y : \{I_{R1}\}$, $v_z : \emptyset$, $i_a : \{I_{Vin}, I_{S1}, I_{L1}, I_{D1}, I_{AMD1}\}$, $v_b : \{V_{VMD1}, V_{C1}, V_{VMVR}, V_{AMIL}\}$, $i_b : \{I_{VMD1}, I_{C1}, I_{VMVR}, I_{AMIL}\}$, $v_a : \{V_{Vin}, V_{S1}, V_{L1}, V_{D1}, V_{AMD1}\}$.

The resistor R_1 is the only non-port element in the boost converter circuit. Thus the last row of equation 4.2 can be written with Ohm's law ($V_{R1} = I_{R1} * R_1$), in which $F_{vy} = 1$, $F_{iy} = -R_1$, and all other sub-matrices are zero.

The tableau form in equation 4.2 can be further simplified by equation 4.3 and 4.4 to form the F matrix [6]. Following these equations, the F matrix of the boost converter circuit is shown in

Table 4.3.

$$\begin{bmatrix} D_{ay} & 0 & 1 & 0 & D_{ab} & 0 \\ 0 & -D_{zb}^t & 0 & 1 & 0 & -D_{ab}^t \\ \hat{F}_{iy} & \hat{F}_{vz} & \hat{F}_{ia} & \hat{F}_{vb} & \hat{F}_{ib} & \hat{F}_{va} \end{bmatrix} \begin{bmatrix} i_y \\ v_z \\ i_a \\ v_b \\ i_b \\ v_a \end{bmatrix} = \mathbf{0} \quad (4.3)$$

$$\begin{aligned} \hat{F}_{iy} &= F_{iy} - F_{iz}D_{zy} \\ \hat{F}_{vz} &= F_{vz} + F_{vy}D_{zy}^t \\ \hat{F}_{ib} &= F_{ib} - F_{iz}D_{zb} \\ \hat{F}_{va} &= F_{va} + F_{vy}D_{ay}^t \\ \hat{F}_{ia} &= F_{ia} \\ \hat{F}_{vb} &= F_{vb} \end{aligned} \quad (4.4)$$

Table 4.3: F Matrix (Full View)

$\mathbf{I}_{R1}$	$\mathbf{I}_{Vin}$	$\mathbf{I}_{S1}$	$\mathbf{I}_{L1}$	$\mathbf{I}_{D1}$	$\mathbf{I}_{AMD1}$	$\mathbf{V}_{VMD1}$	$\mathbf{V}_{C1}$	$\mathbf{V}_{VM-VR}$	$\mathbf{V}_{AM-IL}$	$\mathbf{I}_{VMD1}$	$\mathbf{I}_{C1}$	$\mathbf{I}_{VM-VR}$	$\mathbf{I}_{AM-IL}$	$\mathbf{V}_{Vin}$	$\mathbf{V}_{S1}$	$\mathbf{V}_{L1}$	$\mathbf{V}_{D1}$	$\mathbf{V}_{AMD1}$
0	1	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0
1	0	1	0	0	0	0	0	0	0	0	1	1	-1	0	0	0	0	0
0	0	0	1	0	0	0	0	0	0	0	0	0	-1	0	0	0	0	0
-1	0	0	0	1	0	0	0	0	0	1	-1	-1	0	0	0	0	0	0
-1	0	0	0	0	1	0	0	0	0	0	-1	-1	0	0	0	0	0	0
0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	-1	0
0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	-1	0	1	1
0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	-1	0	1	1
0	0	0	0	0	0	0	0	0	1	0	0	0	0	-1	1	1	0	0
-R1	0	0	0	0	0	0	0	0	0	0	1	0	-1	-1	0	1	0	-1

4.3.3 Extract Linear System Matrix

At this point, observe that the ordering of the elements within the port variable vector is arbitrary. Consequently, the columns are rearranged into the following order [14]:

$$\begin{bmatrix} \mathbf{w} & \tilde{\mathbf{u}} & \mathbf{s} & \mathbf{y} & \dot{\mathbf{x}}^{(0)} & \mathbf{x}^{(0)} & \mathbf{0} & \mathbf{u} \end{bmatrix}^T \quad (4.5)$$

where the constituent vectors are defined as:

$$\begin{aligned}
\mathbf{w} &= \begin{bmatrix} \mathbf{i}_y & \mathbf{v}_z \end{bmatrix}^T \\
\tilde{\mathbf{u}} &= \begin{bmatrix} \mathbf{i}_{E_i} & \mathbf{v}_{J_i} \end{bmatrix}^T \\
\mathbf{s} &= \begin{bmatrix} \mathbf{i}_{ON} & \mathbf{v}_{OFF} \end{bmatrix}^T \\
\mathbf{y} &= \begin{bmatrix} \mathbf{i}_{sc} & \mathbf{v}_{oc} \end{bmatrix}^T \\
\dot{\mathbf{x}}^{(0)} &= \begin{bmatrix} \dot{\mathbf{v}}_C & \dot{\mathbf{i}}_L \end{bmatrix}^T \\
\mathbf{x}^{(0)} &= \begin{bmatrix} \mathbf{v}_C & \mathbf{i}_L \end{bmatrix}^T \\
\mathbf{0} &= \begin{bmatrix} \mathbf{v}_{sc} & \mathbf{i}_{oc} & \mathbf{v}_{ON} & \mathbf{i}_{OFF} \end{bmatrix}^T \\
\mathbf{u} &= \begin{bmatrix} \mathbf{v}_{E_i} & \mathbf{i}_{J_i} \end{bmatrix}^T
\end{aligned} \tag{4.6}$$

The proposed structure explicitly orders the locations as follows: (a) response voltage or current variables of non-port elements; (b) response variables of source ports; (c) switching port variables; (d) output variables; (e) derivatives of all capacitor voltages and inductor currents; (f) the capacitor voltages and inductor currents themselves; (g) zero-valued port variables; and (h) input source values[14, 13, 15].

After re-ordering the system according to this vector structure and applying row-echelon form reduction, the matrix can be expressed as:

$$\begin{bmatrix} \mathbf{T} & \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ \mathbf{0} & \mathbf{U} & \dots & \dots & \dots & \dots & \dots & \dots \\ \mathbf{0} & \mathbf{0} & \mathbf{S} & \dots & \dots & \dots & \dots & \dots \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{Q} & -\mathbf{C}_1 & -\mathbf{C} & \dots & -\mathbf{D} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{M} & -\mathbf{A} & \dots & -\mathbf{B} \end{bmatrix} \begin{bmatrix} \mathbf{w} \\ \tilde{\mathbf{u}} \\ \mathbf{s} \\ \mathbf{y} \\ \dot{\mathbf{x}}^{(0)} \\ \mathbf{x}^{(0)} \\ \mathbf{0} \\ \mathbf{u} \end{bmatrix} = \mathbf{0} \tag{4.7}$$

In (4.7), the blocks $\mathbf{T}$, $\mathbf{U}$, $\mathbf{S}$, and $\mathbf{Q}$ multiply the variable vectors $\mathbf{w}$, $\tilde{\mathbf{u}}$, $\mathbf{s}$, and $\mathbf{y}$, respectively,

and the block $\mathbf{M}$ multiplies the vector of time derivatives $\dot{\mathbf{x}}^{(0)}$.

If the variables of the circuit elements inside the n -port are determined, $\mathbf{T}$ is a nonsingular square block. $\mathbf{Q}$ can be reduced to an identity matrix by elementary row operations [14]. $\mathbf{M}$ is obtainable as a square echelon matrix of order equal to the number of state variables. This block may or may not have zero rows according to the fact that the variables in $\mathbf{x}$ may or may not be linearly dependent. The number of zero rows corresponds to the number of reactive elements for which V_c or I_l are not independent variables.

From 4.7, the state-space iteration is defined as:

$$\mathbf{M}\mathbf{E}\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u} \quad (4.8)$$

with output equation:

$$\mathbf{y} = \mathbf{C}_1\mathbf{E}\dot{\mathbf{x}} + \mathbf{C}\mathbf{x} + \mathbf{D}\mathbf{u} \quad (4.9)$$

where $\dot{x}$ is the derivative of the state variables and $\mathbf{E}$ defines the capacitance and inductance values of the reactive elements in the circuit [11]. $\mathbf{E}$ is typically a diagonal matrix, except when modeling mutual inductance. For the boost converter:

$$\mathbf{E} = \begin{bmatrix} L & 0 \\ 0 & C \end{bmatrix}$$

The linear system matrix of the boost converter circuit (after reordering and row-echelon reduction) is shown in Table 4.4.

Table 4.4: Matrix at Switch On and Diode Off

I_{R1}	I_{Vin}	I_{S1}	V_{D1}	I_{AMD1}	I_{AM-IL}	V_{MD1}	V_{M-VR1}	V_{L1}	I_{C1}	I_{L1}	V_{C1}	V_{S1}	I_{D1}	V_{AMD1}	V_{AM-IL}	I_{VMD1}	I_{VM-VR}	V_{Vin}
1	0	0	0	0	0	0	0	0	0	0	-1/R	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
0	0	1	0	0	0	0	0	0	0	-1	0	0	1	0	0	1	0	0
0	0	0	1	0	0	0	0	0	0	0	1	-1	0	1	0	0	0	0
0	0	0	0	1	0	0	0	0	0	0	0	0	-1	0	0	-1	0	0
0	0	0	0	0	1	0	0	0	0	-1	0	0	0	0	0	0	0	0
0	0	0	0	0	0	1	0	0	0	0	1	-1	0	1	0	0	0	0
0	0	0	0	0	0	0	1	0	0	0	-1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	1	0	0	0	1	0	0	1	0	0	-1
0	0	0	0	0	0	0	0	0	1	0	1/R	0	-1	0	0	-1	1	0

4.4 Handling Topology Changes

As indicated in equation 4.6, the s columns represent current switching port states, where the port's current indicates the switch is ON and the port's voltage indicates the switch is OFF.

During ideal switched network operation, topology changes affect only the s columns. When an ideal switch changes from the ON to the OFF state, the corresponding port ceases to be a voltage port and becomes a current port. This implies an exchange between its port variables in the arrays s and 0 of 4.6, which involves an exchange in corresponding columns of the matrix.

Continuing with the boost converter example, to transition from S1 ON / D1 OFF to S1 OFF / D1 OFF, one must first swap the column corresponding to I_{S1} with the column corresponding to V_{S1} (see Table 4.5). The matrix is then reduced to row-echelon form, as reflected in Table 4.6.

Table 4.5: Matrix With Columns Swapped for Switch S1 OFF and Diode D1 OFF

I_{R1}	I_{Vin}	V_{S1}	V_{D1}	I_{AMD1}	I_{AM1-IL}	V_{VMD1}	V_{VM1-VR}	V_{L1}	I_{C1}	I_{L1}	V_{C1}	I_{S1}	I_{D1}	V_{AMD1}	V_{AM1-IL}	I_{VMD1}	I_{VM1-VR}	V_{Vin}
1	0	0	0	0	0	0	0	0	0	0	-0.2	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	-1	0	1	1	0	0	1	0	0
0	0	-1	1	0	0	0	0	0	0	0	1	0	0	1	0	0	0	0
0	0	0	0	1	0	0	0	0	0	0	0	0	-1	0	0	-1	0	0
0	0	0	0	0	1	0	0	0	0	-1	0	0	0	0	0	0	0	0
0	0	-1	0	0	0	1	0	0	0	0	1	0	0	1	0	0	0	0
0	0	0	0	0	0	0	1	0	0	0	-1	0	0	0	0	0	0	0
0	0	1	0	0	0	0	0	1	0	0	0	0	0	0	1	0	0	-1
0	0	0	0	0	0	0	0	0	1	0	1/R	0	-1	0	0	-1	1	0

Table 4.6: Matrix at Switch Off and Diode Off

I_{R1}	I_{Vin}	V_{S1}	V_{D1}	I_{AMD1}	I_{AM1-IL}	V_{VMD1}	V_{VM1-VR}	V_{L1}	I_{C1}	I_{L1}	V_{C1}	I_{S1}	I_{D1}	V_{AMD1}	V_{AM1-IL}	I_{VMD1}	I_{VM1-VR}	V_{Vin}
1	0	0	0	0	0	0	0	0	0	0	-1/R	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	1	0
0	0	1	0	0	0	0	0	1	0	0	0	0	0	0	1	0	0	-1
0	0	0	1	0	0	0	0	1	0	0	1	0	0	1	1	0	0	-1
0	0	0	0	1	0	0	0	0	0	0	0	0	-1	0	0	-1	0	0
0	0	0	0	0	1	0	0	0	0	0	0	-1	-1	0	0	-1	0	0
0	0	0	0	0	0	1	0	1	0	0	1	0	0	1	1	0	0	-1
0	0	0	0	0	0	0	1	0	0	0	-1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	1	0	1/R	0	-1	0	0	-1	1	0
0	0	0	0	0	0	0	0	0	0	1	0	-1	-1	0	0	-1	0	0

In Table 4.6, the M matrix is no longer an identity matrix as the inductor current is no longer an independent variable when both the switch and diode are OFF. While some dependent and independent variables are visually explicit (a dependent variable has a corresponding zero row

in the M matrix with its pivot falling in the x columns, and an independent variable may correspond to a row with a single non-zero entry), complex circuit topologies often result in rows with multiple non-zero entries, creating ambiguity in variable assignment.

To resolve this consistently, the problem is formulated as a maximum bipartite matching problem. We construct a bipartite graph where one set of nodes represents the ambiguous rows of the M matrix, and the other set represents the candidate state variables (columns) that have non-zero coefficients in those rows. An edge exists between a row and a column if the corresponding matrix entry is non-zero. By defining a cost matrix based on these connections, we utilize the Hungarian algorithm (linear sum assignment) to find a globally optimal assignment that maps each unambiguous row to a unique independent state variable. This guarantees that every independent state is accounted for without conflict, leaving the remaining unassigned variables as dependent states solvable in polynomial time.

4.5 Handling Degenerate Networks and Impulsive Modes

As mentioned in the previous section, certain switch topologies may lead to dependent state variables. For the following analysis, let x_1 represent all **independent state variables** and x_2 represent all **dependent state variables** with M_f defined in equation 4.10.

$$M_f = ME \quad (4.10)$$

Then, equation 4.8 can be rewritten as:

$$\begin{bmatrix} M_{f11} & M_{f12} \\ 0 & 0 \end{bmatrix} \begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} B_1 \\ B_2 \end{bmatrix} u \quad (4.11)$$

This gives rise to two sets of equations:

$$M_{f11}\dot{x}_1 + M_{f12}\dot{x}_2 = A_{11}x_1 + A_{12}x_2 + B_1u \quad (4.12)$$

$$0 = A_{21}x_1 + A_{22}x_2 + B_2u \quad (4.13)$$

When $\mathbf{A}_{22}$ is invertible, $\mathbf{x}_2$ can be solved directly as:

$$x_2 = -A_{22}^{-1}(A_{21}x_1 + B_2u) \quad (4.14)$$

The invertibility of $\mathbf{A}_{22}$ is guaranteed by the fact that the dependent variables are defined as the pivot variables that fell in the x columns during row reduction, which ensures that the corresponding rows in the M matrix are zero rows. This means that the dependent variables are not associated with any time derivatives, and their values are determined solely by the algebraic constraints defined by $\mathbf{A}_{22}$.

In addition, by taking the derivative of equation 4.14, $\dot{x}_2$ can be expressed as:

$$\dot{x}_2 = -A_{22}^{-1}(A_{21}\dot{x}_1 + B_2\dot{u}) \approx -A_{22}^{-1}A_{21}\dot{x}_1 \quad (4.15)$$

assuming the derivative of input $\dot{u}$ is zero for one step.

Substituting equations 4.14 and 4.15 into equation 4.12, the state-space iteration can be reformulated as:

$$(M_{f11} - M_{f12}A_{22}^{-1}A_{21})\dot{x}_1 = (A_{11} - A_{12}A_{22}^{-1}A_{21})x_1 + (B_1 - A_{12}A_{22}^{-1}B_2)u \quad (4.16)$$

which further simplifies to:

$$\begin{aligned} A_{11,\text{new}} &= (M_{f11} - M_{f12}A_{22}^{-1}A_{21})^{-1}(A_{11} - A_{12}A_{22}^{-1}A_{21}) \\ B_{1,\text{new}} &= (M_{f11} - M_{f12}A_{22}^{-1}A_{21})^{-1}(B_1 - A_{12}A_{22}^{-1}B_2) \end{aligned} \quad (4.17)$$

$$\dot{x}_1 = A_{11,\text{new}}x_1 + B_{1,\text{new}}u$$

Plugging equation 4.17 into equation 4.15, the dependent variable $\dot{\mathbf{x}}_2$ can be redefined as:

$$\dot{x}_2 = -A_{22}^{-1}A_{21}A_{11,\text{new}}x_1 - A_{22}^{-1}A_{21}B_{1,\text{new}}u \quad (4.18)$$

Fusing equations 4.17, 4.18, and 4.14, the new state-space iteration of $\dot{\mathbf{x}}$ can be expressed

as:

$$\begin{aligned}
\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} &= A_{new} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + B_{new} u \\
A_{new} &= \begin{bmatrix} A_{11,new} & 0 \\ -A_{22}^{-1} A_{21} A_{11,new} & 0 \end{bmatrix} \\
B_{new} &= \begin{bmatrix} B_{1,new} \\ -A_{22}^{-1} A_{21} B_{1,new} \end{bmatrix} \\
A_{dependent} &= \begin{bmatrix} I, & 0 \\ -A_{22}^{-1} A_{21}, & 0 \end{bmatrix}
\end{aligned} \tag{4.19}$$

where $A_{dependent}$ defines the relationship between dependent and independent state variables.

In similar fashion, the output equation 4.9 can be reformulated as:

$$\begin{aligned}
y &= C_1 E \dot{x} + Cx + Du \\
y &= C_1 E \dot{x} + \begin{bmatrix} C_{01} & C_{02} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + Du \\
y &= C_1 E \dot{x} + C_{new} x + D_{new} u \\
C_{new} &= \begin{bmatrix} C_{01} - C_{02} A_{22}^{-1} A_{21} & 0 \end{bmatrix} \\
D_{new} &= (D - C_{02} A_{22}^{-1} B_2)
\end{aligned} \tag{4.20}$$

which can be further reduced by substituting equation 4.19 as:

$$\begin{aligned}
y &= C_1 E \dot{x} + C_{new} x + D_{new} u \\
y &= Y_{impulse} + Y_{natural} \\
y_{impulse} &= C_1 E A_{new} x_k + C_1 E B_{new} u_k \\
y_{natural} &= C_{new} x_k + D_{new} u_k
\end{aligned} \tag{4.21}$$

4.6 Discrete-Time Implementation

4.6.1 Next State Logic

Given the system is a Linear Time-Invariant (LTI) system during each time step, the next state $\mathbf{x}_{k+1}$ can be calculated as:

$$x_{k+1} = e^{A_{new}\Delta t}x_k + \int_0^{\Delta t} e^{A_{new}\tau} d\tau B_{new}u_k \quad (4.22)$$

The discrete-time update equation 4.22 can be discretized using various numerical integration methods such as Forward Euler, Trapezoidal, or Pade [8].

With that in mind, the discrete-time state-space iteration can be expressed as:

$$x_{k+1} = A_d A_{dependent} x_k + B_d A_{dependent} u_k \quad (4.23)$$

where A_d and B_d are the discretized versions of A_{new} and B_{new} respectively.

The logic for the output at each time step is defined as:

- If external switch toggled or no diode state change: $y_{next} = y_{impulse} + y_{natural}$
- else : $y_{next} = y_{natural}$

4.6.2 Diode Toggle Logic

By using the output equations, it is possible to determine the diode states at each time step. Under normal operation, a diode is ON when the voltage across it is positive, and OFF when the current is positive. However, during the switch transition, the ON/OFF of an external switch may cause dependent states, leading to an impulse response which also affects the diode state. In addition, due to the inaccuracies of the fixed time step and numerical discretization approach, the switch value for the current and next simulation step might be contradicting, causing unnecessary numerical oscillation [9].

While the voltage and current value can be directly obtained from the output equation by looking at the corresponding voltage/current of the diode based on its current state, the impulse

response is calculated using the following equation:

$$\begin{aligned} y_{impulse} &= C_1 E \dot{x} \\ y_{impulse} &= C_1 E \left[A_{dependent} - I \right] x \end{aligned} \quad (4.24)$$

In addition, the prediction for diodes' state at the next time step is given by equation 4.25 using the Forward Euler method:

$$\begin{aligned} y &= (C_1 E A_{new} + C_{new})x + (C_1 E B_{new} + D_{new})u \\ \frac{d}{dt}y &= \dot{y} = (C_1 E A_{new} + C_{new})\dot{x} + (C_1 E B_{new} + D_{new})\dot{u} \\ \dot{y} &\approx (C_1 E A_{new} + C_{new})\dot{x} \\ y_{next} &\approx \dot{y}\Delta t + y \approx (C_1 E A_{new} + C_{new})\dot{x}\Delta t \\ y_{next} &\approx (C_1 E A_{new} + C_{new})(A_{new}x + B_{new}u)\Delta t + y \end{aligned} \quad (4.25)$$

The final diode state decision is made based on the following logic:

- If external switch toggled:

If $V_{D_{impulse}} > 0$: Turn diode ON

Else if $I_{D_{impulse}} > 0$: Turn diode OFF

- Else:

If diode is currently OFF and $V_{D_{natural}} > 0$ and $V_{D_{next}} > 0$: Turn diode ON

Else if diode is currently ON and $I_{D_{natural}} > 0$ and $I_{D_{next}} > 0$: Turn diode OFF

The $V_{D_{impulse}}$ and $I_{D_{impulse}}$ are the voltage and current of the diode defined in equation 4.24, not the $y_{impulse}$ defined in 4.21. The reason is that the impulse response of the diode state is mainly caused by the impulse response of the dependent variables, which is defined as $C_1 E (A_{dependent} - I)x$ in equation 4.24, while the $y_{impulse}$ defined in 4.21 also includes the contribution from independent variables, which is not the reason for causing the potential spike/impulse[3, 15, 13].

$V_{D_{natural}}$ and $I_{D_{natural}}$ are the voltage and current of the diode's natural response defined in equation 4.21, and $V_{D_{next}}$ and $I_{D_{next}}$ are the voltage and current of the diode's next state prediction defined in equation 4.25 as suggested by [9].

Chapter 5

Experiment Setup

This section described the theory and methodology used in realizing the state-space iteration along with experiment setup.

5.1 Principle

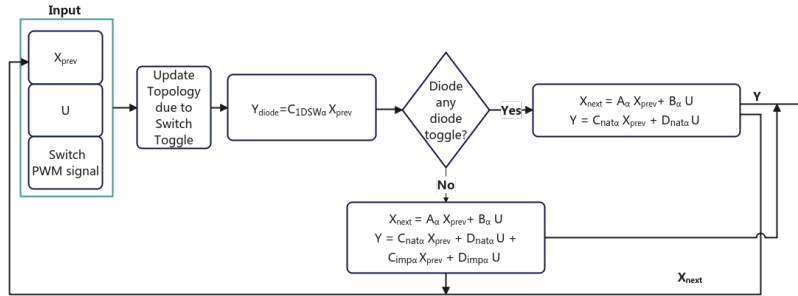


Figure 5.1: Flow chart of the Iteration Process

5.1.1 Algorithm Acceleration

For efficient and fast HIL algorithm, $C_{1DSW\alpha}$ is the matrix formed for calculating diode's $y_{impulse}$, $y_{natural}$, and y_{next} as defined in section 4.6.2, the α represents the current topology of the circuit.

While our HIL algorithm is a customized implementation of the linear state-space iteration method, drawing inspiration from the work of Massarini [13] and exhibiting key similarities to the

algorithms used in commercial HIL platforms such as PLECS [2] and DSPACE [9]. The overall process of one simulation iteration is illustrated in Figure 5.1.

Each simulation run begins by initializing the state vector X_{prev} to all zeros. For every iteration:

1. Retrieve the current input vector U (representing voltages or currents).
2. Update the circuit topology based on changes in switch positions.
3. Perform MVM using $C_{1DSW\alpha}$ to compute diode voltages or currents.
4. If any diode voltage exceeds 0 or current drops below 0, update the diode states and circuit topology accordingly.
5. Calculate the next state vector X_{next} and the output response, including both impulsive and non-impulsive components.

Although each iteration involves relatively lightweight computations—primarily MVM and bitwise logic for topology updates—the critical challenge lies in completing each iteration within strict real-time constraints, particularly when simulating high-frequency switching behavior.

5.1.2 Target Circuit: Half-Bridge LLC Converter

To evaluate the algorithm, we perform a real-time HIL simulation on a half-bridge LLC resonant converter circuit, shown in Figure 5.2. This topology is widely used in power electronics for its simple structure and inherent efficiency advantages, including zero-voltage switching (ZVS) and zero-current switching (ZCS) on the primary switches and secondary rectifiers, respectively [10]. These properties minimize switching losses and enable high-performance operation.

The LLC converter’s resonant behavior leads to non-linear gain characteristics and frequency-dependent dynamics, which make it analytically intractable—especially under real-world conditions such as parasitic effects and time-varying loads. This complexity makes high-resolution, time-domain simulation essential for accurate design and evaluation.

Although we use the half-bridge LLC converter as our demonstration case, the proposed simulation framework is inherently generalizable. Because it is built on modular linear state-space representations and topology-aware matrix updates, our approach can be applied to a broad range

of power electronic systems—including buck, boost, buck-boost, inverter, and multi-level converter circuits. This versatility makes it a promising foundation for real-time simulation and HIL testing across diverse power electronics applications.

5.1.3 Simulation Parameters and Timing Constraints

In our setup, both switches (S1 and S2) operate at 100 kHz with a 50% duty cycle. S2 is phase-shifted by half a clock period relative to S1: S1 is ON during the first half of the cycle, while S2 is ON during the second half. The three-winding transformer is modeled as a mutual inductance with a coupling factor of 0.99.

Following circuit analysis, the system is reduced to a state-space model with 6 state variables ($X \in \mathbf{R}^{6 \times 1}$), 1 input ($U \in \mathbf{R}^{1 \times 1}$), and 5 output variables ($Y \in \mathbf{R}^{5 \times 1}$), including V_{Rout} , $V_{primary}$, V_{L1} , I_{Rout} , and I_{L1} . To achieve accurate results with an oversampling factor of 20 (as required for HIL fidelity), each simulation iteration must be completed in less than $0.5 \mu\text{s}$ [9].

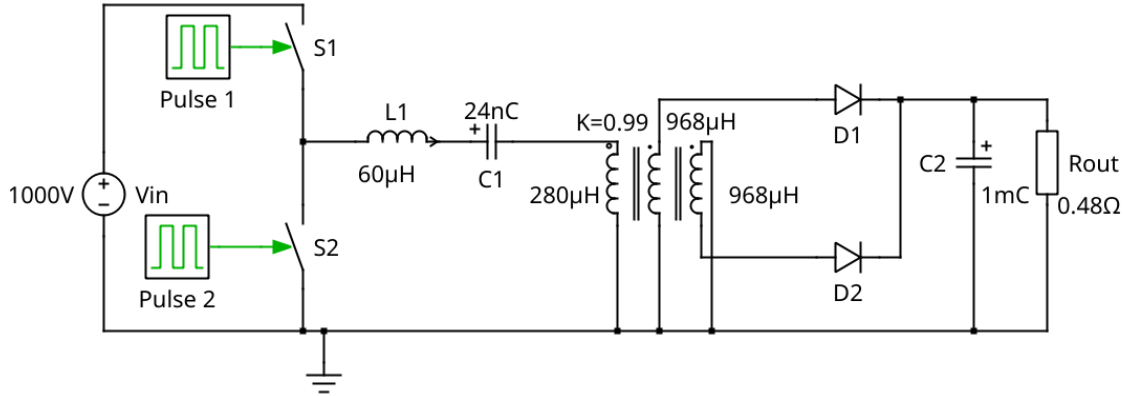


Figure 5.2: Circuit diagram of half-bridge LLC converter.

5.2 Implementation

Due to the relatively small size of the state-space model for the half-bridge LLC converter, our implementation uses only two CTs on the XDNA2 architecture—specifically, *CT0.3* and *CT0.2*. These tiles are arranged in a pipelined configuration, as illustrated in Figures 5.4 and 5.5.

5.2.1 Matrix Vector Multiplication (MVM)

To fully exploit the XDNA2 CT’s support for emulated SIMD FP32 operations, which operate most efficiently on vectors of length 16, all matrices—namely $C_{1DSW\alpha}$, AB_{α} , $CD_{imp\alpha}$, and $CD_{nat\alpha}$ —are stored in column-major order with zero-padding as needed. This memory layout ensures that the data aligns with a stride of 16, enabling efficient vectorized processing, as illustrated in Figure 5.3.

Because XDNA2 does not natively support FP32 MVM, we implement MVM using the built-in fused multiply-add (FMA) intrinsic. Specifically, each matrix-vector product is computed as a linear combination of the matrix’s columns, with each column having a length of 16. This FMA-based approach not only takes advantage of the SIMD pipeline for parallel multiplication but also improves numerical accuracy. By accumulating each column’s result in a dedicated register until the final output is written, we avoid intermediate rounding and truncation that would otherwise occur in row-by-row computations.

5.2.2 Diode Toggle

Unlike the HIL algorithms implemented in dSPACE [9] and PLECS [3], which apply advanced logic minimization techniques—such as the Quine-McCluskey algorithm—to derive the minimal disjunctive normal form (MDNF) for determining diode toggling and circuit topology transitions, our implementation adopts a more straightforward approach.

Specifically, during each simulation iteration, a diode is turned **on** if it experiences either:

- a positive impulse response (resulting from a recent switch event), or
- a steady-state voltage greater than zero ($V > 0$).

Conversely, a diode is turned **off** if it experiences either:

- a negative impulse response, or
- a steady-state current less than zero ($I < 0$).

as described in section 4.6.2.

To efficiently implement this logic on the XDNA2 architecture, we leverage its SIMD vector unit. For every group (or stride) of 16 diode outputs Y_{diode} , we generate two bitmasks—one indicating

where values are greater than zero, and another where values are less than zero. These bitmasks are then used to determine the diode toggle status via parallel bitwise operations. This strategy allows us to encode diode switching behavior entirely in bitwise logic, enabling high-throughput, vectorized execution on the AI Engine.

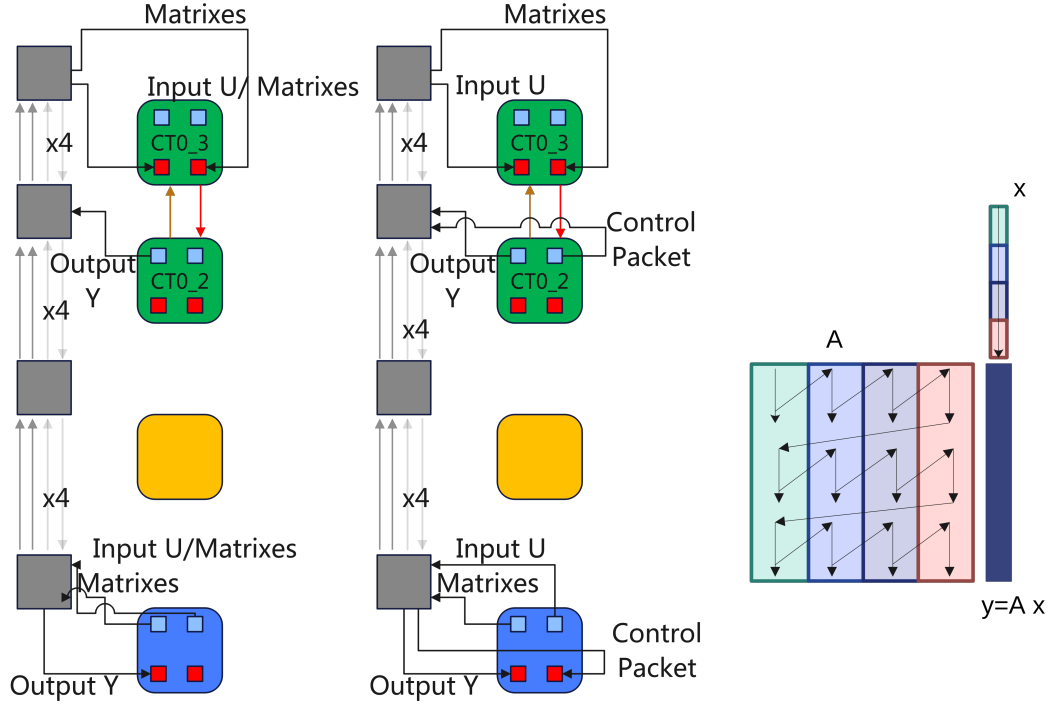


Figure 5.3: Data flow of the two implementations (left) and column-major MVM illustration (right).

5.2.3 Data Transfer Architecture and Buffering Strategy

Each switch box in the XDNA2 architecture supports at least four AXI4-Stream connections in each of the four cardinal directions (north, south, east, and west), enabling flexible tile-to-tile communication. In particular, the vertical direction (south to north) supports up to six AXI4-Stream paths, allowing for high-throughput data movement across vertically aligned compute tiles.

Despite this rich interconnect fabric, each CT is equipped with only two DMA engines. These DMA units are responsible for reading from and writing to memory and operate at a clock frequency of 1.8 GHz with a 64-bit data width. The peak theoretical data transfer bandwidth per

tile is computed as:

$$\text{Bandwidth}_{\text{max}} = 2 \times 1.8 \text{ GHz} \times 64 \text{ bits} = 28.8 \text{ Gbps}$$

This hardware configuration is illustrated in Figure 5.3, where red boxes indicate stream-to-memory (S2MM) ports and blue boxes denote memory-to-stream (MM2S) ports associated with each DMA engine.

To enable parallelism between data movement and computation, we adopt a double-buffering strategy for both input and output streams. Specifically, two independent sets of memory buffers are allocated for each simulation iteration. While one buffer set is actively accessed by the DMA engine for reading or writing, the compute tile simultaneously performs calculations using the other buffer set. This alternating, or “ping-pong,” buffer scheduling allows for continuous data throughput and minimizes idle time in the pipeline.

Some commercial HIL platforms employ advanced strategies to reduce latency—both algorithmically and at the FPGA hardware level. However, these approaches often assume that all circuit topology matrices can be preloaded into on-chip memory, which is generally valid in FPGA-based systems. In contrast, AIE architectures impose different memory constraints. In this work, we explicitly evaluate both scenarios: one where all required matrices fit entirely within local AIE memory, and another where matrices must be fetched dynamically at runtime. This broader scope allows our analysis to account for memory-bound limitations and offers practical insights for deploying real-time simulations on spatial AI architectures.

5.2.4 Pipeline Architecture

While pipelined simulation architectures for AIEs have been previously explored in domains such as Finite-Difference Time-Domain (FDTD) methods for stencil-based computations [23], this work is the first to apply such an architecture to real-time, state-space-based circuit simulation on an AIE.

5.2.4.1 Matrix Cached Implementation

In the matrix-cached implementation, all topology-specific simulation matrices— $C_{1DSW\alpha}$, AB_{α} , $CD_{\text{impulse}\alpha}$, and $CD_{\text{natural}\alpha}$ —are preloaded into the local memory bank of tile CT0.3. Mean-

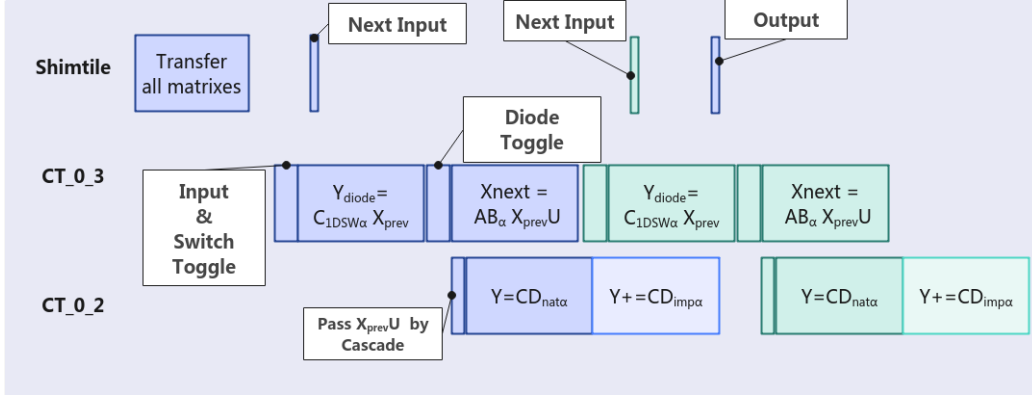


Figure 5.4: Matrix Cached Pipeline

while, the input and output buffers are allocated in the local memory of tile $CT0.2$. This memory arrangement is illustrated in Figure 5.3.

Figure 5.4 shows the pipelined execution across $CT0.3$ and $CT0.2$, where the blue shading represents iteration 0 and the green shading represents iteration 1.

At the start of each iteration, both MM2S (memory-to-stream) ports on $CT0.3$ are configured to read the matrices AB_{α} , $C_{1DSW\alpha}$, $CD_{impulse\alpha}$, and $CD_{natural\alpha}$ corresponding to the current circuit topology. Simultaneously, a pair of S2MM (stream-to-memory) and MM2S ports on the shim tile are configured to handle the transfer of input and output data between the host system and tile $CT0.2$.

The execution flow proceeds as follows:

1. $CT0.3$ first performs a MVM between $C_{1DSW\alpha}$ and X_{prev} , which is the previous state vector.
2. Once complete, $CT0.3$ forwards X_{prev} to $CT0.2$ via the cascade stream interconnect (indicated by the red arrow in Figure 5.3). It then continues to compute the MVM between AB_{α} and X_{prev} .
3. Upon receiving X_{prev} , $CT0.2$ computes MVMs using $CD_{natural\alpha}$ and, if needed, $CD_{impulse\alpha}$ to produce the non-impulsive and impulsive components of the output.

Due to the compute-bound nature of FP32 MVM operations on XDNA2 (as discussed in the Experiment section), it is sufficient for $CT0.2$ to fetch the $CD_{impulse\alpha}$ and $CD_{natural\alpha}$ matrices directly from $CT0.3$'s memory without incurring significant performance penalties.

Additionally, our use of double-buffering for input and output streams allows the shim tile to concurrently:

- load the next iteration’s input data, and
- store the previous iteration’s output results.

This enables seamless overlap between data movement and computation across pipeline stages, ensuring efficient simulation throughput under real-time constraints.

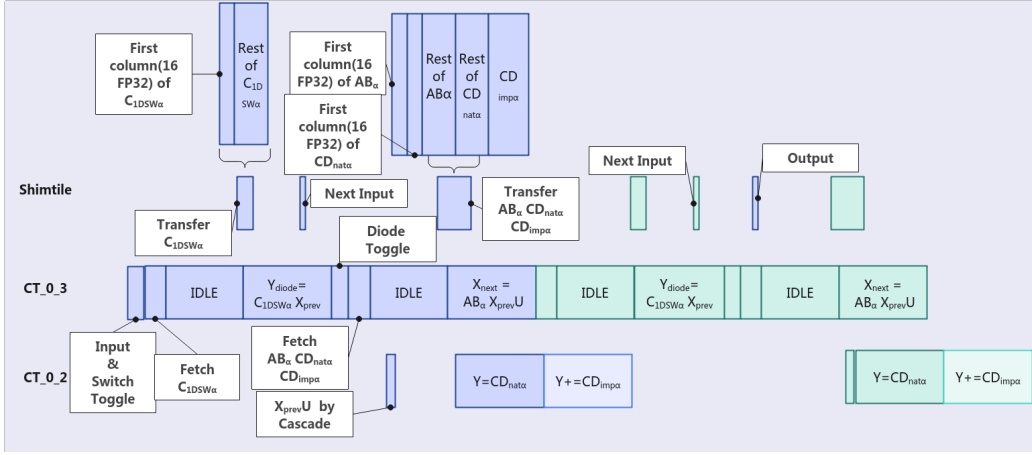


Figure 5.5: Runtime Matrix Fetching Pipeline

5.2.4.2 Runtime Matrix Fetching

Although AIE is intended for data-flow programming paradigm, the device includes built-in control-packet features that allow CT to send control instructions to shimtile, as indicated in the extra control-packet path on figure 5.3. The runtime matrix fetching implementation allocated a double buffer for inputs and outputs on $CT0.2$. Meanwhile, $CT0.3$ allocates buffers for current topology of AB_α , C_{1DSW_α} , $CD_{impulse_\alpha}$, and $CD_{natural_\alpha}$ matrices.

Figure 5.5 shows the pipeline structure of $CT0.3$ and $CT0.2$ for the simulation. At each iteration, after fetching current iteration’s input and switch state, $CT0.3$ sends a control packet to shimtile to fetch current C_{1DSW_α} . $CT0.3$ then stalls until its DMA receives the first column of C_{1DSW_α} . Similarly, after the diode toggle logic, which might change the topology again, $CT0.3$ then sends another control packet to shimtile to fetch AB_α , $C_{impulse_\alpha}$, and $CD_{natural_\alpha}$ matrices. Given the compute-bound(see Experiment section) nature of the MVM operation, the matrices

AB_α , $C_{impulse\alpha}$, and $C_{1DSW\alpha}$ were pre-arranged prior to simulation. This arrangement specifically involved placing the first column of AB_α , followed by the first column of $C_{1DSW\alpha}$, and then the remaining columns of AB_α , $C_{1DSW\alpha}$, and $C_{impulse\alpha}$, which gives both $CT0.3$ and $CT0.2$ nearly the same stalling time.

Chapter 6

Results

6.1 Experiment Setup

All experiments in this work were conducted on a compact, commercially available MiniPC: the GMKtec NUCBOX EVo-X1. This system is equipped with an AMD Ryzen AI 9 HX 370 processor (codenamed *Strix*), which includes 32 usable AIE tiles and 32 GB of DDR5 system memory.

The AIE tiles are part of AMD’s XDNA2 architecture and serve as the primary compute resources for our simulation kernels. All tests were executed with the AIE clock frequency set to 1.8 GHz, corresponding to the processor’s turbo performance mode. This configuration reflects a realistic embedded deployment scenario and provides a meaningful benchmark for evaluating the system’s ability to meet real-time simulation requirements.

6.2 Accuracy Evaluation

To rigorously assess the accuracy of our HIL simulation, we compare the output waveforms generated by both the Matrix Cached and Runtime Matrix Fetching implementations on the AIE against two well-established reference models.

The first reference is a custom C++ implementation of the simulation algorithm, using single-precision FP32 arithmetic. This implementation adopts a Padé 0/3 discretization method with $y = 0$, as described in [8], and serves as a software ground truth. The second reference is the output from a commercial PLECS model. This PLECS simulation is configured for code generation

with a fixed time-step iteration mode, providing a reliable industry-standard baseline.

To ensure a fair comparison, all three simulations—the two AIE-based implementations, the C++ model, and the PLECS model—are configured with an identical iteration step size of $0.5 \mu\text{s}$.

Figures 6.1 and 6.2 illustrate the resulting waveforms. The voltage output from our AIE-based HIL simulation aligns very closely with that of the PLECS model, confirming the validity of the proposed pipeline. Minor discrepancies are observed when comparing the AIE results (green and red traces) to the C++ simulation, but these differences are expected due to the precision limitations of emulated FP32 operations on the AIE. Importantly, the deviations remain within an acceptable range and do not compromise the overall simulation fidelity.

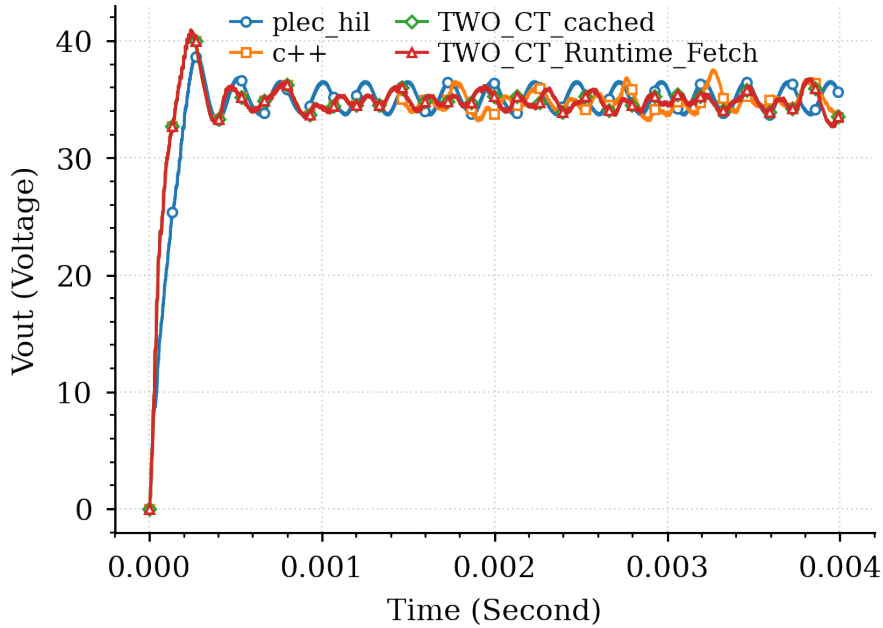


Figure 6.1: Output Voltage of Half-Bridge-LLC at x20 oversampling iteration frequency.

6.3 Performance analysis

To obtain a detailed understanding of the AIE kernel’s execution characteristics, we utilized the built-in tracing tools provided by the AMD AIE development environment. These tools enable cycle-level profiling, allowing precise measurement of operation-level latencies. Due to trace memory limitations, Table 6.1 presents the average cycle counts for key operations, aggregated over the first

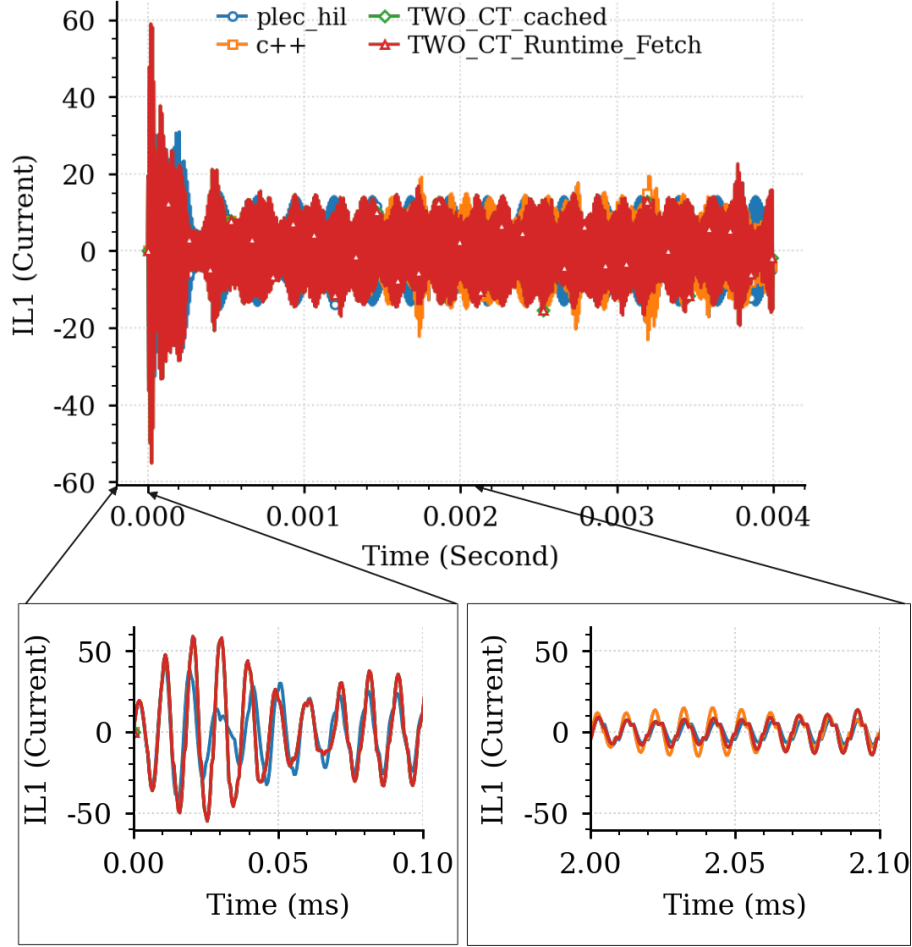


Figure 6.2: L1 Current of Half-Bridge-LLC at x20 oversampling iteration frequency.

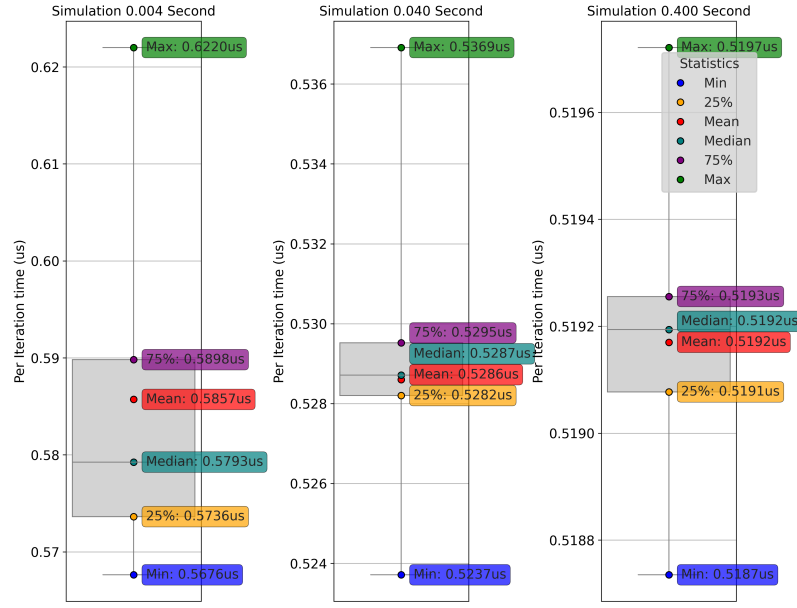
30 cycles of simulation iterations.

A critical observation is that MVM using FP32 arithmetic on the AIE is compute-bound rather than memory-bound. For instance, an MVM operation involving a 16×7 matrix and a 7×1 vector took approximately 455 cycles to complete. In contrast, the theoretical time to transfer this matrix over a single DMA stream is just 56 cycles, computed as:

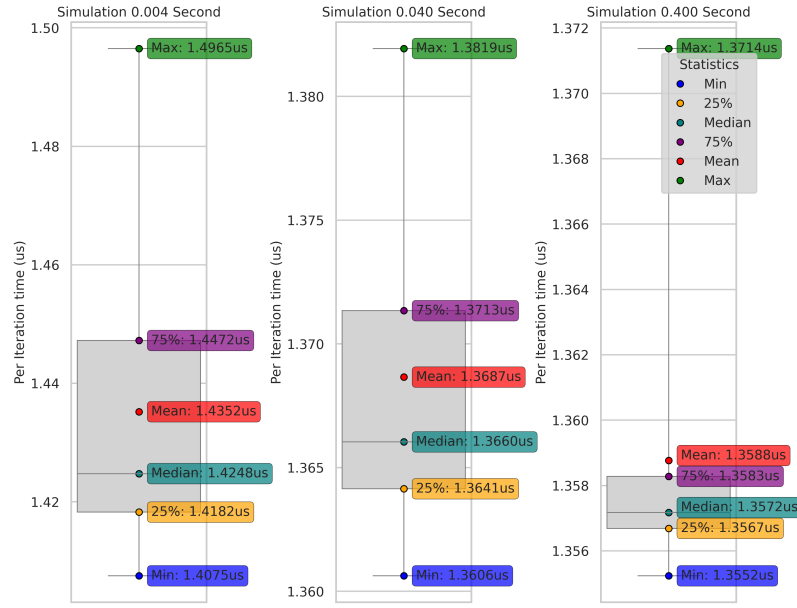
$$\frac{7 \times 16 \times 4}{8} = 56 \text{ cycles},$$

where 4 represents the number of bytes per FP32 value, and the DMA width is 8 bytes.

Figures 6.3a and 6.3b show the performance profiles for the Matrix Cached and Runtime Matrix Fetching implementations, respectively. Each experiment was executed 50 times. The per-



(a) Matrix Cached Implementation



(b) Runtime Matrix Fetching Implementation

Figure 6.3: Box and Whisker plot showing iteration times for (a) Matrix Cached and (b) Runtime Matrix Fetching implementations. 50 runs per configuration.

iteration time was calculated as the total execution duration divided by the number of iterations. Notably, the box plots include all data points—including outliers—to accurately reflect worst-case performance. This is crucial in HIL systems, where the maximum iteration time often determines whether real-time constraints are met.

We observed that simulations executed over shorter time spans (e.g., 0.004 seconds) reported higher average iteration times compared to longer runs (e.g., 0.4 seconds). This discrepancy arises from fixed overheads—such as AIE task launch latency and initial matrix transfer setup—which are more prominent during shorter runs. As the simulation continues, these startup costs become amortized, yielding a more stable and representative measure of core iteration performance.

Among the two strategies, the Matrix Cached implementation consistently delivered iteration times near the 0.5 μ s real-time threshold, with minimal variance across runs. This indicates strong temporal predictability, a desirable property for deterministic HIL systems.

In contrast, while the Runtime Matrix Fetching strategy also exhibited stable behavior across runs, its average iteration time was significantly longer. Further profiling using the Linux **perf** tool and low-level AIE logs revealed that the performance bottleneck was due to frequent cache misses during dynamic matrix loads. These misses introduced stalls of approximately 450 cycles per matrix fetch, explaining the overall performance degradation observed in this configuration.

Table 6.1: TABLE I: Average Cycle Breakdown Per Iteration for Key Simulation Operations.

Operation	Average Cycles
MVM	455
Input & Switch Toggle	16
Diode Toggle	16
Fetch Matrix Operation	170
Idle Time (Waiting Data For Runtime Fetching Mode)	397–455

Chapter 7

Conclusions and Discussion

7.1 Summary of Contributions

This thesis represents, to the best of our knowledge, the pioneering exploration of utilizing the AMD AI Engine (AIE)—a Neural Processing Unit (NPU) architecture—for the real-time Hardware-in-the-Loop (HIL) simulation of power electronics. While NPUs are traditionally optimized for deep learning inference, this work successfully demonstrates their potential applicability to the stiff, high-frequency differential equations inherent in power converter simulation.

The primary contributions of this work are threefold: the systematic derivation of a graph-theoretic modeling framework, the novel implementation of state-space iteration on a spatial dataflow architecture, and a rigorous performance analysis of memory management strategies on the XDNA2 hardware.

7.2 Methodological Framework

A critical prerequisite for this implementation was the formulation of a generalized methodology for realizing the state-space matrices. Moving beyond ad-hoc circuit equations, we adopted a systematic approach based on graph theory and Massarini’s method.

By translating the circuit topology into an incidence matrix and subsequently deriving the cutset matrix, we established a robust pipeline that automates the extraction of the state-space system. This methodology effectively handles the complexities of ideal switching, including the

management of dependent state variables and the detection of Dirac impulses caused by instantaneous topological changes. This theoretical rigor ensures that the simulation engine is not limited to the specific Half-Bridge LLC converter tested here but is extensible to a wide class of power electronic topologies.

7.3 Implementation Strategies and Performance

We investigated two distinct architectural strategies to map this mathematical framework onto the AIE’s localized memory hierarchy:

- **Matrix Cached Implementation:** By pre-loading all topology-specific matrices into the local memory of the Compute Tiles, this approach eliminated external memory latency. It achieved near real-time performance for a 100 kHz switching frequency with highly deterministic iteration times. This validates the AIE’s raw compute capability for HIL tasks, provided the model fits within the localized storage limits.
- **Runtime Matrix Fetching:** This approach attempted to overcome memory capacity limits by dynamically fetching matrices via DMA at every time step. However, our analysis revealed significant performance bottlenecks driven by driver-level cache misses and DMA initialization overheads. This finding highlights a critical current limitation in the AIE software stack for low-latency, small-payload transfers typical of HIL applications.

7.4 Discussion on Architecture and Scalability

The experimental results highlight a fundamental trade-off when utilizing current NPU architectures for scientific computing. The XDNA2 architecture is heavily optimized for low-precision (int8) deep learning throughput. Consequently, our reliance on emulated FP32 support resulted in the system being compute-bound, with Matrix-Vector Multiplication (MVM) consuming the majority of the cycle budget.

Our analysis of spatial parallelism revealed that simply distributing the state vector across multiple Compute Tiles (e.g., scaling from 2 to 4 tiles) yielded diminishing returns. In this algebraic decomposition approach, the communication overhead required to synchronize the full state vector

between tiles negated the benefits of parallel processing. This suggests that for tightly coupled systems like power converters, standard matrix splitting within a small cluster of tiles is inefficient.

However, scalability can be re-evaluated through the lens of circuit partitioning (diakoptics). Rather than splitting a single matrix algebraically, the circuit topology itself can be decomposed into multiple sub-circuits, with each tile simulating a specific physical partition. This approach potentially reduces inter-tile communication overhead significantly, as tiles would only need to exchange boundary conditions—typically a small subset of voltage or current values—rather than the entire state vector.

Furthermore, while this specific implementation utilized only two Compute Tiles, the AIE architecture offers a vast array of available tiles. This spatial abundance presents an opportunity to shift the focus from minimizing the latency of a single simulation to maximizing the throughput of multiple simulations. This architecture is ideally suited for “batch” processing, where dozens of independent simulation instances run in parallel—a capability that is particularly valuable for generating synthetic data for Reinforcement Learning (RL) training or performing Monte Carlo failure analyses.

7.5 Final Remarks and Future Work

Despite the challenges imposed by emulated precision and driver latency, this work demonstrates that the AMD AIE is a functional platform for real-time circuit simulation. It offers a compelling alternative to FPGAs for applications where design flexibility, rapid compilation, and software-defined workflows are prioritized over nanosecond-level timing precision.

To fully realize the potential of the AIE for Hardware-in-the-Loop applications, future research should focus on three key areas:

- **Topological Partitioning:** Future implementations should investigate splitting the circuit into coupled sub-circuits assigned to individual tiles, thereby minimizing communication bandwidth requirements compared to monolithic matrix operations.
- **Software Stack Optimization:** Further optimization of the underlying software stack is essential, specifically to reduce DMA initialization overhead and improve cache coherency for the small-payload transfers typical of runtime matrix fetching.

- **Massive Parallelism for AI:** Future work should leverage the multi-core nature of the AIE to run massive parallel simulations. This could serve as a high-speed verification environment for Reinforcement Learning agents, utilizing the NPU not just for the neural network policy, but for the physical environment simulation itself.

Bibliography

- [1] Advanced Micro Devices, Inc. AMD XDNA™ Architecture. <https://www.amd.com/en/technologies/xdna.html>, 2025. Accessed June 10, 2025.
- [2] J. H. Allmeling and W. P. Hammer. Plectra—piece-wise linear electrical circuit simulation for simulink. In *Proceedings of the IEEE 1999 International Conference on Power Electronics and Drive Systems (PEDS'99)*, volume 1, pages 355–360, 1999.
- [3] Jost Allmeling, Niklaus Felderer, and Min Luo. High fidelity real-time simulation of multi-level converters. In *2018 International Power Electronics Conference (IPEC-Niigata 2018 - ECCE Asia)*, pages 2199–2203, 2018.
- [4] Handy Fortin Blanchette, Tarek Ould-Bachir, and Jean Pierre David. A state-space modeling approach for the fpga-based real-time simulation of high switching frequency power converters. *IEEE Transactions on Industrial Electronics*, 59(12):4555–4567, 2012.
- [5] R. Champagne, L.-A. Dessaint, H. Fortin-Blanchette, and G. Sybille. Analysis and validation of a real-time ac drive simulator. *IEEE Transactions on Power Electronics*, 19(2):336–345, 2004.
- [6] L. O. Chua and P. M. Lin. *Computer-Aided Analysis of Electronic Circuits*. Prentice-Hall, 1975.
- [7] J. A. Fortanet Capetillo. Exploration of the amd ryzen npu for real-time signal processing: Real-time imaging of lofar station data. Master’s thesis, Delft University of Technology, 2025. Master’s thesis, Delft University of Technology.
- [8] Axel Kiffe. *Echtzeitsimulation Leistungselektronischer Schaltungen für die Hardware-in-the-Loop-Simulation*. Universitätsverlag der TU Berlin, Berlin, 2018.
- [9] Axel Kiffe, Thorben Hoffstadt, and Frank Puschmann. Modeling approach for real-time capable oversampling models applied to a fast switching llc resonant converter. In *2021 23rd European Conference on Power Electronics and Applications (EPE'21 ECCE Europe)*, pages P.1–P.11, 2021.
- [10] Dong-Kwan Kim, Cheol-O Yeon, Jae-Hyun Kim, Yeonho Jeong, and Gun-Woo Moon. Llc resonant converter with high voltage gain using auxiliary lc resonant circuit. In *2015 9th International Conference on Power Electronics and ECCE Asia (ICPE-ECCE Asia)*, pages 1505–1512, 2015.
- [11] Yitong Li, Timothy C. Green, and Yunjie Gu. Descriptor state space modeling of power systems. *IEEE Transactions on Power Systems*, 39(4):5495–5508, 2024.
- [12] LLVM Project. Multi-Level IR Compiler Framework. <https://mlir.llvm.org>, 2024. Accessed June 10, 2025.

- [13] A. Massarini and U. Reggiani. Computer-aided time-domain large-signal analysis of networks with switches. In *Proceedings of IEEE International Symposium on Industrial Electronics*, volume 2, pages 567–572, 1996.
- [14] A. Massarini and U. Reggiani. An efficient algorithm for the formulation of state and output equations for networks with ideal switches. In *ISCAS 2001. IEEE International Symposium on Circuits and Systems*, pages 521–524, 2001.
- [15] A. Massarini, U. Reggiani, and M.K. Kazimierczuk. Analysis of networks with ideal switches by state equations. *IEEE Transactions on Circuits and Systems I: Fundamental Theory and Applications*, 44(8):692–697, 1997.
- [16] Caio R. D. Osório, Milos Miletic, Jovan Zelic, Dusan Majstorovic, and Ognjen Gagrca. Advancements on real-time simulation for high switching frequency power electronics applications (invited paper). In *2021 21st International Symposium on Power Electronics (Ee)*, pages 1–6, 2021.
- [17] P. Pejovic and D. Maksimovic. A method for fast time-domain simulation of networks with switches. *IEEE Transactions on Power Electronics*, 9(4):449–456, 1994.
- [18] Alejandro Rico, Satyaprakash Pareek, Javier Cabezas, David Clarke, Baris Ozgul, Francisco Barat, Yao Fu, Stephan Münz, Dylan Stuart, Patrick Schlangen, Pedro Duarte, Sneha Date, Indrani Paul, Jian Weng, Sonal Santan, Vinod Kathail, Ashish Sirasao, and Juanjo Noguera. Amd xdna™ npu in ryzen™ ai processors. *IEEE Micro*, pages 1–10, 2024.
- [19] André Rösti and Michael Franz. Unlocking the amd neural processing unit for ml training on the client using bare-metal-programming tools. In *2025 IEEE 33rd Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, pages 271–271, 2025.
- [20] Endri Taka, Andre Roesti, Joseph Melber, Pranathi Vasireddy, Kristof Denolf, and Diana Marculescu. Striking the balance: Gemm performance optimization across generations of ryzen ai npus. *arXiv preprint arXiv:2512.13282*, 2025.
- [21] Xifan Tang, Edouard Giacomini, Giovanni De Micheli, and Pierre-Emmanuel Gaillardon. Fpga-spice: A simulation-based architecture evaluation framework for fpgas. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 27(3):637–650, 2019.
- [22] Xilinx. IRON AIE Programming Guide. <https://github.com/Xilinx/mlir-aie>, 2024. Accessed June 10, 2025.
- [23] Zhenyu Xu, Miaoxiang Yu, Yazhe Zhang, Jillian Cai, Qing Yang, and Tao Wei. Tile-level pipeline for linear scalable stencil computation on amd ai engines. In *Proceedings of the 2025 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, pages 172–178, 2025.